

# Optimization and Heuristics Project 2 Report

*Optimization and Heuristics (18275)*

Matteo Oliver Bekink (575997)

Aleksander Nowak (576069)

October 2025

## Contents

|                                               |           |
|-----------------------------------------------|-----------|
| <b>List of Symbols</b>                        | <b>2</b>  |
| <b>Introduction</b>                           | <b>2</b>  |
| <b>Part 1: Solving a CSP Problem</b>          | <b>3</b>  |
| Problem Statement . . . . .                   | 3         |
| CSP Modeling . . . . .                        | 3         |
| <b>Part 2: Solving a Search Problem</b>       | <b>6</b>  |
| Problem Statement . . . . .                   | 6         |
| CSP Modeling . . . . .                        | 6         |
| <b>Part 3: Analysis of results</b>            | <b>8</b>  |
| Results of Solving a CSP Problem . . . . .    | 8         |
| Results of Solving a Search Problem . . . . . | 11        |
| <b>A Terminal outputs</b>                     | <b>13</b> |

## List of Symbols

| Symbol   | Definition                   | Domain                  |
|----------|------------------------------|-------------------------|
| $x_{ij}$ | White Disk Decision Variable | $\mathbb{B} : \{0, 1\}$ |
| $y_{ij}$ | Black Disk Decision Variable | $\mathbb{B} : \{0, 1\}$ |
| $C_I$    | Constraint I,II,...          |                         |

## Introduction

In this report we shall describe the problems encountered as part of the Second Project for the class "Heuristics and Optimization", completed under the guidance of our supreme lecturer Vidal Alcaraz Saiz.

We hope that you find this description to your liking and are generous with your grading.

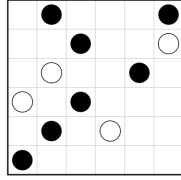
## Part 1: Solving a CSP Problem

### Problem Statement

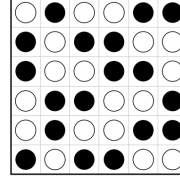
Let there be a square grid dimensions  $n \times n$  composed of  $n^2$  cells. Each cell **must** take either a black or white disc such that:

1. No position must be left empty
2. The number of white and black disks in each row, and in each column, **must** be equal
3. It is not possible to place more than two discs of the same color consecutively in the same row or column.

The objective of the CSP solver is to take an initial state of the problem and to find all the solutions that satisfy the CSP problem.



(a) Initial state of CSP problem



(b) Final state of CSP problem

Figure 1: An example of an initial configuration and one of its solutions

### CSP Modeling

#### Decision Variables

A key skill in CSP modeling is being able to convert a real world problem, such as this problem, into a set of binary variables, which can be interpreted as on/off switches. Although the behavior of each cell is binary, ie. the cell is either occupied or not, we must ensure that the cell can differentiate between black and white disks, hence, this contradicts that binary behavior.

An alternative approach is to generate a **binary** variable for each cell and color disk. Each cell is therefore treated as two binary variables that dictate whether or not their respective colored disk is on the cell or not.

$$x_{ij} = \begin{cases} 0 \rightarrow \text{no white disk in cell } (i, j) \\ 1 \rightarrow \text{white disk in cell } (i, j) \end{cases} \quad \forall i, j \in [0, n)$$

$$y_{ij} = \begin{cases} 0 \rightarrow \text{no black disk in cell } (i, j) \\ 1 \rightarrow \text{black disk in cell } (i, j) \end{cases} \quad \forall i, j \in [0, n)$$

Note that the grid is zero-indexed. Although this method is easier to implement, it adds an extra constraint to the CSP problem :

(IV) *No two disks can occupy the same cell.*

### Constraints

Modeling the constraints of a CSP problem involves representing them in form of a logical or arithmetic set of operations between decision variables. By convention, the logical constraints are written in conjunctive normal form (**CNF**) and the arithmetic constraints written as inequalities ( $\leq, \geq$ ) or equalities ( $=$ ).

**Constraint I** can be initially modeled as a set of OR ( $\vee$ ) statements between each decision variable at each cell.

$$C_I(x_{ij}, y_{ij}) = x_{ij} \vee y_{ij} \quad \forall i, j \in [0, n)$$

Although this does satisfy **Constraint I**, it contradicts **Constraint IV**. Hence an alternative approach would be to use an XOR ( $\oplus$ ) statement and to write it in CNF form as follows.

$$\begin{aligned} C_I(x_{ij}, y_{ij}) &= x_{ij} \oplus y_{ij} = (x_{ij} \wedge \bar{y}_{ij}) \vee (\bar{x}_{ij} \wedge y_{ij}) \\ &= (x_{ij} \vee y_{ij}) \wedge (\bar{x}_{ij} \vee \bar{y}_{ij}) \quad \forall i, j \in [0, n) \end{aligned} \quad (1)$$

The result is a constraint that is only true when the decision variables in a particular cell are not the same. Coincidentally, this covers both **Constraint I** and **Constraint IV** simultaneously.

**Constraint II** is actually an arithmetic constraint as it involves counting the number of disks in each row and column. Modeling is as simple as counting the number of white and black disks in each row or column and ensuring they are the same as such.

$$C_{II_r}(i) : \sum_{j=0}^n x_{ij} = \sum_{j=0}^n y_{ij} \quad \forall i \in [0, n) \quad (2)$$

$$C_{II_c}(j) : \sum_{i=0}^n x_{ij} = \sum_{i=0}^n y_{ij} \quad \forall j \in [0, n) \quad (3)$$

Note that the constraint is split into two sub constraints, each representing the constraint along a row or a column.

**Constraint III** is the trickiest of all to implement as it cannot be represented as a single logical statement but rather as a sequence of clauses. Modelling such a problem can be done easiest by trying to model the valid states of the problem directly and then simplifying the logical statement into CNF form.

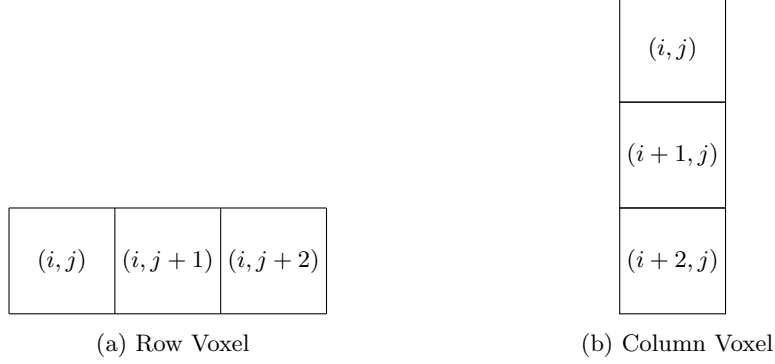


Figure 2: Voxels of  $(i, j)$

Figure 2 is a simplified schematic of the row and columns of the voxels for the cell  $(i, j)$ . Note that in Figure 2, cell  $(i, j + 1)$  in 2a corresponds to  $(i + 1, j)$  in 2b. It is formatted in this way to simplify boundary cases.

However, given there are 32 permutations of the cell  $(i, j)$  and its neighbors (excl. diagonals), rather than modelling the cases where **Constraint III** is satisfied, we can model the cases where it is not satisfied as they are fewer. Note that, if the row voxel 2a is fully filled in, regardless of how the column voxel is filled in, the constraint is not satisfied and vice versa. Hence, we the state of the row voxel is decoupled from the state of the column voxel. Similar to Equation 2 and 3, we can split **Constraint III** into sub constraints. We can start by writing out the cases where **Constraint III** is *not* met for each color disk and then applying a not ( $\neg$ ) to the logical statement and simplifying into CNF form:

#### Row Voxel Constraint

$$\begin{aligned}
 C_{III_r}^{(x)}(x_{i,j}) &= \neg(x_{i,j} \wedge x_{i,j+1} \wedge x_{i,j+2}) \\
 &= \neg x_{i,j} \vee \neg x_{i,j+1} \vee \neg x_{i,j+2} \quad \forall i \in [0, n), \quad j \in [0, n-2) \\
 C_{III_r}^{(y)}(y_{i,j}) &= \neg(y_{i,j} \wedge y_{i,j+1} \wedge y_{i,j+2}) \\
 &= \neg y_{i,j} \vee \neg y_{i,j+1} \vee \neg y_{i,j+2}
 \end{aligned}$$

#### Column Voxel Constraint

$$\begin{aligned}
 C_{III_c}^{(x)}(x_{i,j}) &= \neg(x_{i,j} \wedge x_{i+1,j} \wedge x_{i+2,j}) \\
 &= \neg x_{i,j} \wedge \neg x_{i+1,j} \wedge \neg x_{i+2,j} \quad \forall i \in [0, n-2), \quad j \in [0, n) \\
 C_{III_c}^{(y)}(y_{i,j}) &= \neg(y_{i,j} \wedge y_{i+1,j} \wedge y_{i+2,j}) \\
 &= \neg y_{i,j} \wedge \neg y_{i+1,j} \wedge \neg y_{i+2,j}
 \end{aligned}$$

Note that the domains for  $i$  and  $j$  are shortened to ensure that the cells in the voxels do not exceed the boundaries of the problem dimensions  $n \times n$ .

## Part 2: Solving a Search Problem

### Problem Statement

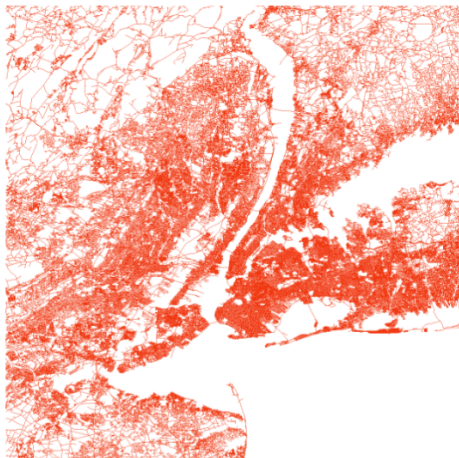


Figure 3: Map of NYC network

The *9th DIMACS Shortest-Path Challenge* is a well known challenge that invites anyone to design a solution to the classical search problem: getting from point A to point B following real world maps following the *shortest path*. Simple as it may sound, its a classical problem that is continuously being improved upon.

### Algorithm Implementation

#### A\* Implementation

The map above is represented by way of a graph. The nodes of the graph represent individual locations and a list of edges is provided which describe the possible direct connections from one node to others as well as their cost.

The nature of this problem means that the  $A^*$  path-finding algorithm is a perfect fit. When using this solution, it is of utmost importance to use the most efficient data structures available for the open and closed lists.

For the *open list* we need a data structure which does the following things fast:

- Extraction of smallest element - operation performed once per step
- Insertion of new element - operation performed multiple times per step

The *min-heap* (priority queue) fulfills the above criteria by having an  $O(1)$

time complexity for extracting the minimum value, and a  $O(\log N)$  time complexity for the insertion of new elements.

The *closed list* requires the following operations to be performed quickly:

- Insertion of new element - operation performed once per step
- Membership testing - operation performed multiple times per step

The *set* is perfect for the above, having an  $O(1)$  time complexity for both the membership testing and insertion.

The A\* algorithm also requires us to define a heuristic which it can use to determine the node to expand at each iteration. Each of them needs to be admissible (the value of the heuristic can never overestimate the real cost of a node) and consistent (the total estimated cost of a node never decreases along any path).

In order to better understand how the choice of heuristic affects the performance of the A\* algorithm, we will be using several different methods to calculate them:

- Haversine formula
- Euclidean distance

### Dijkstra Implementation

In our code, the brute force algorithm we will be comparing A\* against is the Dijkstra algorithm.

It is implemented by using an A\* solver with a heuristic which always takes the same value, usually  $h(n) = 0$ .

### Other optimizations

There are several other, smaller optimizations which we have added to our code in an attempt to increase the performance of our search algorithms. These are:

- Epsilon heuristic - in the Dijkstra algorithm,  $h(n) = \epsilon, \epsilon = 1$ . This accounts for the fact that the minimum distance from one node can be no smaller than 1.
- Comparison tiebreaks - in the case where two nodes have the same estimated cost, the one with a lower real cost will be treated as if it had the lower value.
- Early goal detection - if the node selected for expansion is the goal node, we do not perform node expansion and instead stop the algorithm early.

## Part 3: Analysis and Results

In this section, we will explore the outcomes of the modeling cases for both types of problems described in this report. Several examples of solutions will be shown and verified for accuracy. An analysis of the solving runtime for each problem will be given together with answering a problem-specific question to be answered.

### Results of Solving a CSP Problem

The problem was modelled and solved in Python using the `python-constraint` library. The script takes in a input file `input.in` and returns a terminal output containing the number of solutions found. Furthermore one of the solutions is then stored in an `output.out` file together with the initial problem. The input file contains the initial state represented with either '.', '**X**' or '**O**' that symbolize empty cells, black disks or white disks respectively. By running the following command, the script runs and solves the problem described in `input.in`.

```
python parte-1.py input.in output.out
```

Before delving into runtime analysis, the first step is to verify the integrity of the solutions that the problem model generates. To do so a test problem (`test1.in`) must be solved both hand and with the solver to verify the solutions.

```
+---+---+---+---+
|   | X |   | O |
| O |   |   |   |
|   | X |   |   |
|   |   |   | X |
+---+---+---+---+
```

(a) Test 1 Initial State

```
+---+---+---+---+
| X | X | O | O |
| O | O | X | X |
| O | X | X | O |
| X | O | O | X |
+---+---+---+---+
```

(b) Test 1 Solution

Figure 4: Initial and Solution states of Test 1

Upon visual inspection Figure 4b follows all the constraints and looks a valid solution to the input state. Solving by hand shows that this simple test problem does indeed have 3 solutions in total (A) hence confirming that the script runs correctly.

The next problem that was solved was the problem given in the project file `default.in`:

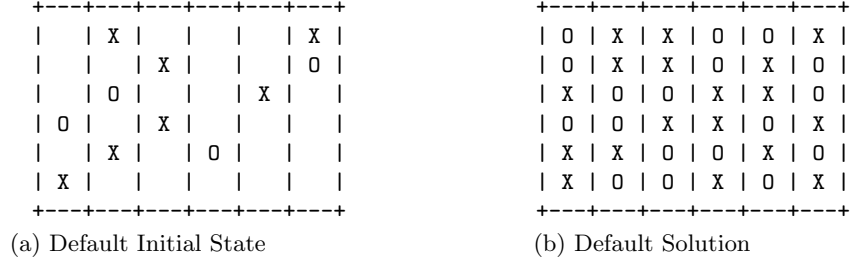


Figure 5: Initial and Solution states of Default

The results from this test (A) indicate that there are 9 solutions in total, which aligns with the outputs given in the project file. Important to note is that the runtime has increased with respect to the solution of `test1.in` and the number of **Constraint III** constraints has increased more than the number of **Constraint I/IV** constraints.

By repeating this process of scaling up to larger and larger problems, we can analyse how the number of constraints increase with the size of the problem and how to run times may be affected. In a sort-of linear regression, we can identify the time-constraining constraint. Before that however, an interesting question to ask:

*Does the proportion of pre-inputted cells affects the execution time?*

. Our prediction is that the behaviour acts quasi-exponential with a decaying behaviour. This is because the number of cells that could have either a black or white cell decreases, thus the number of total (incl. invalid) permutations decreases. In other words the algorithm searches for less solutions. To answer this question, a series of five problems, all  $(6 \times 10)$  was setup with a varying proportion of input-determined variables. In this way, we control how many constraints there are, as they are dimension dependant. The results are as follows:

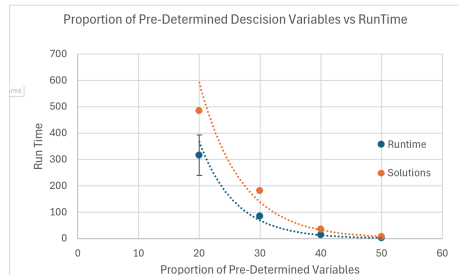


Figure 6: Relationship between number of pre-determined variables and runtime

From Figure 6, it is clear that the behaviour is exponential. The data points was capped at 20% propotion because the run-time was excessively long. The experiment was perfomed five times on each input file and the error was computed and plotted aswell. Note that the number of solutions to each problem also decays exponentially as you increase the proportion of defined variables. **In Figure 6 the number is solutions is scaled down by 1/100.** Therefore, the answer to the question *if the proportion of pre-inputted cells affects the execution time* is a resounding yes, and, this matched predictions made before-hand aswell.

Now we can return to understanding which constraint may be the time-constraining constraint in our problem. If we *assume* that each constraint takes an equal amount of computational time to compute, then clearly **Constraint III** becomes the limiting factor as the dimensions of the problem increases. For the sake of explanation, *assume* that the problem is of dimensions  $n \times n$ . Then the relationship for the number of constraints for each constraint can be modeled as the following equations:

$$f_I(n) = n^2 \quad \forall n \in \mathbb{Z}_0 \quad (4)$$

$$f_{II}(n) = 2n \quad \forall n \in \mathbb{Z}_0 \quad (5)$$

$$f_{III}(n) = \begin{cases} 4n(n-2), & n > 2 \\ 0, & 0 \leq n \leq 2 \end{cases} \quad (6)$$

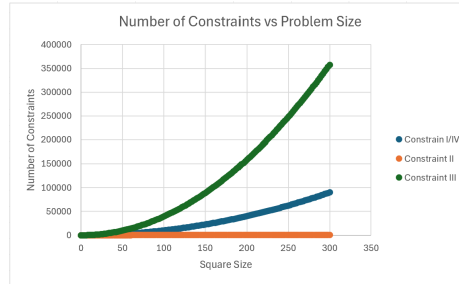


Figure 7: Number of constraints as a function of the square problem dimensions  $n$

Clearly **Constraint III** is the time limiting constraint. The main cause for this is the fact that for each column and row voxel, there are two constraints generated, each for the black and white disks. Consequentially, a potential improvement would be to find an alternative method for modeling which reduces the number of constraints generated and thus the computational time. it should be noted that the likely cause of extended runtime is due to the algorithm searching for solutions within the total permutations.

## Results of Solving a Search Problem

### Experiment setup

For each test, a set of start/goal nodes was created where the nodes were located within a certain distance range of one another.

This set of nodes was then given to each solver who were timed on how long it took each of them to solve all of the supplied problems. Only the time used running the algorithm is counted.

By using this method, we were able to compare each solver to one another on the same set of randomly generated problems of a certain (estimated) difficulty.

### Base comparison

In this example, we generated a set of 50 pairs of nodes located at least 20 km apart from one another. The nodes were sampled from the *USA-road-d.BAY* map from the 9th DIMACS challenge.

|                | Dijkstra | A* (haversine) | A* (euclidean) |
|----------------|----------|----------------|----------------|
| Time [s]       | 540.676  | 631.703        | 1189.071       |
| Nodes expanded | 8406298  | 7887083        | 7886602        |

Table 2: Results of base experiment (50 pairs)

The above results show that Dijkstra achieves the best results despite expanding around 6.5% more nodes. This is due to the overhead introduced to A\* by the necessary heuristic calculations.

While the haversine method doesn't seem to require many more computational steps, the square root calculations needed to generate the Euclidean distance have a significant negative impact on the final result, almost doubling the runtime.

### Increasing the scale

*Disclaimer:* The authors attempted to run these experiments on the Central USA map, but their laptops ran out of memory while trying to load the data from the files.

Dijkstra proved to be the superior algorithm above due to the negligible decrease in expanded nodes provided by A\*. Is this due to the scale of the problem, or simply due to how the graphs in the DIMACS challenge are laid out? In this experiment, we will increase the scale of the problem tenfold to see if this changes Dijkstra's relative performance.

The nodes in this experiment were sampled from the *USA-road-d.LKS* map, which contains 2,758,119 nodes compared to the previous maps' 321,270. The minimum distance between node pairs has been increased to 200 km, and the number of pairs has been lowered to 5.

The *euclidean* heuristic isn't being testing here, as the above experiment proved that it performs much worse than the *haversine*.

|                | Dijkstra | A* (haversine) |
|----------------|----------|----------------|
| Time [s]       | 787.068  | 1106.581       |
| Nodes expanded | 9501789  | 9261338        |

Table 3: Results of the second experiment (5 pairs)

The above results demonstrate that the number of expanded nodes per problem has increased 10 times compared to the previous experiment, yet Dijkstra not only maintains, but extends its advantage over A\*! It now performs 28.8% faster!

This could be due to the overly optimistic nature of the heuristic used in A\* (the haversine formula assumes straight-line distance between points), which leads it to overlook the actual best path in favor of ones which look better initially, but end up leading to dead ends.

The above, coupled with the need to compute a heuristic function at each step, cause the Dijkstra algorithm to outperform it.

## A Terminal outputs

```
1 Time taken to solve: 0.005707502365112305 seconds
2 Total solutions found: 3
3 [[ 'X' 'X' 'O' 'O' ]
4  [ 'O' 'O' 'X' 'X' ]
5  [ 'O' 'X' 'X' 'O' ]
6  [ 'X' 'O' 'O' 'X' ]]
7 Output saved to .\out_files\test1.out
8
9 CONSTRAINTS
10 same_cell_const: 16
11 no_three_horizontal: 16
12 no_three_vertical: 16
13 num_var_const_row: 4
14 num_var_const_col: 4
```

Listing 1: Output of Test 1

```
1 Time taken to solve: 0.027596712112426758 seconds
2 Total solutions found: 9
3 [[ 'O' 'X' 'X' 'O' 'O' 'X' ]
4  [ 'O' 'X' 'X' 'O' 'X' 'O' ]
5  [ 'X' 'O' 'O' 'X' 'X' 'O' ]
6  [ 'O' 'O' 'X' 'X' 'O' 'X' ]
7  [ 'X' 'X' 'O' 'O' 'X' 'O' ]
8  [ 'X' 'O' 'O' 'X' 'O' 'X' ]]
9 Output saved to .\out_files\default.out
10
11 CONSTRAINTS
12 same_cell_const: 36
13 no_three_horizontal: 48
14 no_three_vertical: 48
15 num_var_const_row: 6
16 num_var_const_col: 6
```

Listing 2: Output of Default